

AI Model Advisor

A RAG-powered assistant to understand, compare and choose AI chatbots

Project Documentation

Contents

1. Project Overview
2. Features
3. Tech Stack
4. System Architecture and Workflow
5. Knowledge Base
6. Core Concepts Explained
7. LLM and API Configuration
8. Token Usage
9. Rate Limits and Error Handling
10. Code Walkthrough
11. How to Run the Project
12. Security
13. Limitations and Future Scope
14. Glossary

1. Project Overview

AI Model Advisor is a web application that answers questions about six leading AI chatbots: **ChatGPT, Claude, Gemini, DeepSeek, Grok and Perplexity**. It uses **Retrieval-Augmented Generation (RAG)**, so every answer is based on a knowledge base built from the official documentation and pricing pages of each provider, and every answer shows its sources.

Problem statement

There are many AI chatbots, and each one has different models, prices, API features and strengths. Students, developers and businesses find it hard to decide which one to use. A normal chatbot may give outdated or made-up (hallucinated) information about prices and models.

Objectives

- Answer questions about the 6 chatbots using only verified documentation.
- Compare any two chatbots side by side in a fair way.
- Recommend the best chatbot or model for a user's specific use case.
- Show the sources of every answer to build trust and reduce hallucination.
- Run completely free: local embeddings and a free LLM API.

2. Features

Mode / Feature	What it does
Ask	The user asks any question (models, API, pricing, strengths). The app retrieves the 5 most relevant chunks and the LLM answers only from them.
Compare	The user picks two chatbots and a focus (Pricing, Coding, API, Context window, Free tier, Web search, etc.). The app retrieves 4 chunks for each chatbot separately and returns a comparison table, key differences and a verdict.
Recommend	The user describes a use case (budget, language, task). The app retrieves 10 chunks and returns the best choice with reasons, two alternatives and points to keep in mind.
Source citations	Each answer card shows the source files as tags and a 'View retrieved context' section with the exact chunks given to the LLM.
Example questions	One-click example buttons in Ask and Recommend fill in the question and run it.
Intro video	A short intro video plays at the top of the app when it opens.
Model selector	The sidebar lets the user choose between 4 LLMs as a fallback.

3. Tech Stack

Component	Tool used	Why it was chosen
Programming language	Python	Standard language for AI and data projects.
User interface	Streamlit	Builds a web app in pure Python, quick to develop.
RAG framework	LangChain	Ready-made building blocks for loading, splitting, embedding, searching and calling LLMs.
Text splitter	RecursiveCharacterTextSplitter	Splits text at paragraph, then line, then word boundaries so sentences are not broken.
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (HuggingFace)	Free, small (about 90 MB), runs locally on a laptop, 384-dimension vectors.
Vector database	FAISS (by Meta)	Free, very fast similarity search, runs locally in memory.
LLM provider	Groq API	Free tier, very fast responses, hosts strong open models.
Default LLM	GPT-OSS 120B (open-weight model by OpenAI)	Most capable free model available on Groq.

4. System Architecture and Workflow

The project has two phases: **indexing** (done once when the app starts) and **querying** (done for every question).

Phase 1: Indexing (build the knowledge base)

Step	Stage	What happens
1	Load	Read all .txt files from the data/ folder.
2	Label	Attach metadata to each document: bot (chatbot name) and source (file name).
3	Chunk	Split documents into chunks of 800 characters with 120 characters overlap.
4	Embed	Convert each chunk into a 384-number vector using all-MiniLM-L6-v2.
5	Store	Save all vectors in a FAISS vector store (cached, built only once).

Phase 2: Querying (answer a question)

Step	Stage	What happens
1	Question	The user types a question or picks a mode.
2	Retrieve	FAISS finds the top-k most similar chunks (k = 5, 4+4 or 10).
3	Augment	A prompt is built: role + rules + retrieved chunks + question.
4	Generate	The prompt is sent to the LLM through the Groq API; the LLM writes the answer.
5	Display	The answer is shown in a card with source tags and the retrieved context.

```
data/*.txt -> Load + metadata -> Chunk (800 / 120) -> Embeddings (MiniLM) -> FAISS
|
User question -> similarity_search (top-k) <-----+
              -> Prompt (rules + context + question) -> Groq API (LLM) -> Answer + sources
```

5. Knowledge Base

The knowledge base is a set of plain text files in the **data/** folder. Each file was compiled from the official documentation and pricing pages of the provider (the source links are written inside each file). Data is current as of **25 September 2026**.

File	Content
chatgpt.txt	OpenAI: overview, GPT-6 Astra / Sol / Luna, pricing, API basics, strengths, weaknesses.
claude.txt	Anthropic: Fable 5.1, Opus 5.5, Sonnet 5, Haiku 4.5, pricing, 1M context, API basics.
gemini.txt	Google: Gemini 3.1 Pro, 3.x Flash models, Live model, free tier, API basics.
deepseek.txt	DeepSeek: deepseek-flash, deepseek-v4-pro, peak/off-peak pricing, OpenAI-compatible API.
grok.txt	xAI: Grok 4.7, Voice API, Imagine API, pricing, 500K context.
perplexity.txt	Perplexity: Sonar family, request fees, built-in web search and citations.
overview_comparison.txt	All 6 compared: summary table, best for what, price ranking, cost example.

Statistic	Value
Number of files	7
Total size	about 26,700 characters (about 6,700 tokens)
Number of chunks	48
Average chunk size	about 140 tokens
Vector size	384 numbers per chunk

Adding a new chatbot: add a new file (for example `mistral.txt`) to the `data/` folder and add one line to the `BOTS` dictionary in `app.py`. The cards, dropdowns and search update automatically.

6. Core Concepts Explained

6.1 RAG (Retrieval-Augmented Generation)

RAG is a technique where the AI first **retrieves** relevant information from our own documents, then that information is **added (augmented)** to the prompt, and finally the LLM **generates** the answer using it. Without RAG, an LLM answers only from what it learned during training, which may be old or wrong. With RAG, answers come from up-to-date documents and can show their sources.

6.2 Chunking

Chunking means splitting a large document into small pieces called **chunks**. Small chunks make search more precise, because the app can find the exact part that contains the answer instead of a whole file.

- **chunk_size = 800** characters: big enough to hold one complete idea (for example, one model's pricing).
- **chunk_overlap = 120** characters: the end of one chunk is repeated at the start of the next, so a sentence cut at the border is not lost.
- The **Recursive** splitter first tries to split at paragraphs, then lines, then words, so sentences stay whole.

6.3 Embeddings

An embedding is a list of numbers that represents the **meaning** of a text. The model `all-MiniLM-L6-v2` turns every chunk into 384 numbers. Texts with similar meaning get similar numbers, so "How much does Claude cost?" and "Claude pricing" are found as similar even though the words are different. The embedding model runs locally, so it costs nothing.

6.4 Vector Database (FAISS)

FAISS (Facebook AI Similarity Search) stores all the embeddings and can very quickly find the ones closest to a new question's embedding. In this project FAISS runs in memory on the laptop.

6.5 Similarity Search and k

Similarity search converts the user's question into an embedding and returns the **k** chunks whose embeddings are closest to it. The value of **k** controls how much context the LLM receives.

Mode	k	Reason
Ask	5	Enough context for one focused question, keeps the prompt small.
Compare	4 + 4	Four chunks for each chatbot, retrieved separately (see metadata filtering).
Recommend	10	Needs information about several chatbots to choose the best one.

6.6 Metadata and Metadata Filtering

Metadata is extra information attached to each chunk. Every chunk carries bot (e.g. "claude") and source (e.g. "claude.txt"). It is used in two ways:

- **Filtering in Compare mode:** `filter={"bot": a}` searches only chatbot A's chunks, and the same for B. Without the filter, all top chunks could come from one chatbot and the comparison would be unfair.
- **Source citations:** the source file name is shown under every answer.

6.7 LLM (Large Language Model)

An LLM is an AI model trained on a very large amount of text. It understands a prompt and generates text by predicting the next word again and again. In this project the LLM writes the final answer from the retrieved chunks. The embedding model *finds* information; the LLM *writes* the answer.

6.8 API and API Key

An **API** (Application Programming Interface) is the way our code talks to another service over the internet. Large LLMs need powerful servers, so they run on Groq's servers, and our app sends the prompt to them through the **Groq API**. The **API key** is a secret password that identifies the user, so Groq can allow access and track usage limits.

6.9 Tokens

A **token** is a small piece of text that an LLM reads or writes. It can be a whole word or part of a word.

- Rule of thumb for English: 1 token $\approx$ 4 characters $\approx$ 3/4 of a word; 100 tokens $\approx$ 75 words.
- Example: "I love ChatGPT" $\rightarrow$ "I" | " love" | " Chat" | "GPT" = 4 tokens.
- **Input tokens:** everything sent to the LLM (rules + chunks + question).
- **Output tokens:** the answer the LLM writes. Output tokens are usually more expensive.
- LLM APIs charge per token and limit how many tokens can be used, so fewer tokens means faster and cheaper answers.

6.10 Context Window

The context window is the maximum number of tokens an LLM can read at once (for example, 1 million tokens for recent Claude models and 500K for Grok 4.7). RAG keeps our prompts small (about 900 to 1,600 input tokens), far below any limit.

6.11 Temperature

Temperature controls how creative or random the LLM's output is, usually on a scale from 0 to 1 (or higher).

Temperature	Behaviour	Good for
0 to 0.3 (low)	Focused, consistent, factual	Facts, Q&A, RAG
0.4 to 0.7 (medium)	Balanced	General chat

Temperature	Behaviour	Good for
0.8 and above (high)	Creative, more random	Stories, brainstorming

This project uses **temperature = 0.2**, because answers about prices and models must be accurate and should stay the same if the same question is asked again.

6.12 Prompt and Rules

Every request sends a structured prompt to the LLM: a role ("You are an expert assistant..."), a set of **rules**, the retrieved **context**, and the **question**. The rules are:

- Use **ONLY** the information in the **CONTEXT**. Do not use outside knowledge.
- If the answer is not in the context, say: "Sorry, this information is not in my knowledge base."
- Always reply in English.
- Use clean Markdown only (headings, bullets, tables). **NEVER** use HTML tags.
- Keep table cells short. Put code in separate code blocks below tables.

6.13 Hallucination and How It Is Reduced

Hallucination is when an AI confidently gives false or made-up information. This project reduces it in four ways:

- The rules force the LLM to answer only from the retrieved context.
- The LLM must say "not in my knowledge base" when the answer is missing (test: "Who won the IPL 2025?").
- Low temperature (0.2) keeps answers factual.
- Sources and the retrieved chunks are shown, so every answer can be checked.

6.14 Caching and Session State

- **st.cache_resource**: Streamlit re-runs the whole script on every click. Caching makes the knowledge base (loading, chunking, embedding) build only once, so the app stays fast.
- **st.cache_data**: caches the intro video data.
- **st.session_state**: the app's memory. It keeps each answer on screen when the user switches tabs or clicks other buttons.

7. LLM and API Configuration

Setting	Value
API provider	Groq API (accessed through LangChain's ChatGroq)
Default model	openai/gpt-oss-120b
Backup models	llama-3.3-70b-versatile, openai/gpt-oss-20b, llama-3.1-8b-instant
Temperature	0.2
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (local, no API)
External APIs used	Only one: Groq API

Model	Made by	Size	Notes
GPT-OSS 120B	OpenAI (open-weight)	120 billion parameters	Default; most capable
Llama 3.3 70B	Meta	70 billion parameters	Strong general model
GPT-OSS 20B	OpenAI (open-weight)	20 billion parameters	Faster, lighter
Llama 3.1 8B	Meta	8 billion parameters	Fastest, simplest answers

Important: these models are not made by Groq. They are open models made by OpenAI and Meta. Groq only *hosts* them on its fast servers and gives access through its API.

Why multiple models?

- **Fallback:** each model has its own free-tier limit. If one hits the limit or fails, the user switches in the sidebar.
- **Choice:** a bigger model gives better answers; a smaller model gives faster answers.
- **Demonstration:** the same question can be compared across models.

8. Token Usage

Token counts below are estimated from the actual knowledge base (average chunk $\approx$ 140 tokens, prompt rules $\approx$ 120 tokens). Output size depends on how long the LLM's answer is.

Mode	Chunks sent	Input tokens (approx.)	Output tokens (approx.)	Total per answer
Ask	5	$\sim$ 900	$\sim$ 300	$\sim$ 1,200
Compare	8 (4 + 4)	$\sim$ 1,300	$\sim$ 500	$\sim$ 1,800
Recommend	10	$\sim$ 1,600	$\sim$ 400	$\sim$ 2,000

Why RAG saves tokens: the whole knowledge base is about 6,700 tokens. Sending all of it with every question would make each request about 5 to 7 times larger, slower and costlier. RAG sends only the most relevant chunks.

Cost

The project costs **nothing to run**: Groq's free plan covers the LLM, and embeddings plus FAISS run locally.

9. Rate Limits and Error Handling

The app itself has no question limit. The limits come from **Groq's free plan** (for GPT-OSS models, per model):

Limit	Value	Meaning for this app
Tokens per minute (TPM)	8,000	About 4 to 6 questions per minute
Tokens per day (TPD)	200,000	About 100 to 160 questions per day

Limit	Value	Meaning for this app
Requests per minute (RPM)	30	Token limit is reached first
Requests per day (RPD)	1,000	Token limit is reached first

If a limit is crossed, Groq returns error **429 Too Many Requests**. The app catches the error with try/except, shows a message instead of crashing, and the user can wait a minute or choose another model. Other safeguards:

- If no data files are found, the app shows a clear error and stops.
- If no API key is entered, the app shows a 'Get started in 3 steps' guide and stops.
- If the same chatbot is picked twice in Compare, the app shows a warning.

Limits can change; the latest values are on console.groq.com/docs/rate-limits.

10. Code Walkthrough (app.py)

Part / function	Lines (approx.)	Purpose
Imports	1-18	Load Streamlit, LangChain components, FAISS, HuggingFace embeddings, ChatGroq.
Config (BOTS, RULES, examples)	20-59	Chatbot details (single source of truth), LLM rules, example questions, page setup.
CSS styles	61-133	Visual design only (dark theme, cards, tabs, buttons).
build_store()	138-151	Load files, add metadata, chunk, embed, store in FAISS. Cached.
format_context()	153-154	Join retrieved chunks into one text, each with its source.
run_llm()	156-161	Send the prompt to the LLM; catch errors safely.
render_answer()	163-178	Show the answer card, source tags and retrieved context.
intro_player()	183-210	Autoplay the intro video, with fallbacks.
Sidebar	215-233	API key input, model selector, video toggle, knowledge base stats.
Header and hero	238-262	Navbar, title, stats and chatbot cards (HTML).
Guards and setup	267-288	Stop if no data or key; create the LLM; set up session_state.
Ask tab	291-315	Retrieve 5 chunks, build prompt, generate, display.
Compare tab	318-352	Filtered retrieval (4 + 4), comparison prompt, display.
Recommend tab	355-386	Retrieve 10 chunks, recommendation prompt, display.

Key code: building the knowledge base

```
@st.cache_resource
def build_store():
    docs = []
    for path in sorted(glob.glob(os.path.join(DATA_DIR, "*.txt"))):
        key = os.path.splitext(os.path.basename(path))[0].lower()
        text = open(path, encoding="utf-8").read().strip()
        docs.append(Document(page_content=text,
                             metadata={"bot": key, "source": os.path.basename(path)}))
    chunks = RecursiveCharacterTextSplitter(chunk_size=800,
                                           chunk_overlap=120).split_documents(docs)
    store = FAISS.from_documents(chunks, HuggingFaceEmbeddings(model_name=EMBED_MODEL))
    return store, len(docs), len(chunks)
```

Key code: the RAG query (Ask mode)

```
chunks = store.similarity_search(q, k=5) # Retrieve
prompt = f"{role}\n{RULES}\nCONTEXT:\n{format_context(chunks)}\n"
        f"QUESTION: {q}\nANSWER:" # Augment
answer = llm.invoke(prompt).content # Generate
```

Key code: metadata filtering (Compare mode)

```
ca = store.similarity_search(query, k=4, filter={"bot": a}) # only chatbot A
cb = store.similarity_search(query, k=4, filter={"bot": b}) # only chatbot B
```

Key code: LLM setup

```
llm = ChatGroq(model=llm_model, api_key=api_key, temperature=0.2)
```

11. How to Run the Project

Project structure

```
ai-model-advisor/
├── app.py           # Streamlit app + RAG pipeline
├── requirements.txt # Python libraries
├── README.md       # Short project description
├── intro.mp4       # Intro video
├── .streamlit/config.toml # Dark theme settings
└── data/           # Knowledge base (7 .txt files)
```

Steps

```
# 1. Install libraries (run once)
pip install -r requirements.txt

# 2. Start the app
streamlit run app.py

# 3. In the browser, paste your free Groq API key in the sidebar
# (get it from console.groq.com/keys)
```

Run the two commands on **separate lines**. The first run downloads the embedding model (about 90 MB), so it takes a minute.

12. Security

- The API key box in the sidebar is a password field, so the key is shown as dots.

- The key can also be set as an environment variable, so it never appears on screen:
`$env:GROQ_API_KEY="your_key"` (Windows PowerShell), then `streamlit run app.py`.
- The key is **never** stored in any project file, so it is safe to share or submit the code.
- Never put the key in GitHub, screenshots, videos or chat messages. Delete and recreate the key if it is ever exposed.

13. Limitations and Future Scope

Limitations

- The data is static: when prices or models change, the text files must be updated manually.
- Only 6 chatbots are covered.
- Answer quality depends on the LLM and on how complete the knowledge base is.
- The free API plan has rate limits (about 5 questions per minute).

Future scope

- Automatically scrape the official documentation every week to keep data fresh.
- Add a cost calculator with sliders (users, messages per day).
- Measure answer quality with an evaluation tool such as RAGAS.
- Add more chatbots such as Mistral and Llama.
- Add a real-time talking avatar (for example, HeyGen LiveAvatar) that speaks the answers.
- Support questions in Hindi.

14. Glossary

Term	Simple meaning
RAG	Retrieve relevant information, add it to the prompt, then let the LLM generate the answer.
LLM	Large Language Model: an AI that understands and writes text.
API	The way code talks to another service over the internet.
API key	A secret password that identifies the user to the API.
Token	A small piece of text (about 4 characters) that an LLM reads or writes.
Temperature	Controls how creative or random the LLM's answer is. Low = factual.
Context window	The maximum number of tokens an LLM can read at once.
Chunk	A small piece of a document (here about 800 characters).
Chunk overlap	Characters shared between two neighbouring chunks (here 120).
Embedding	A list of numbers that represents the meaning of a text.
Vector database	A database that stores embeddings and finds similar ones quickly (FAISS).
Similarity search	Finding the chunks whose meaning is closest to the question.
k	How many chunks are retrieved for a question.
Metadata	Extra labels attached to each chunk (chatbot name, file name).
Hallucination	When an AI gives false or made-up information.
Prompt	The full instruction sent to the LLM (role, rules, context, question).
Rate limit	The maximum usage allowed by the API in a given time.
Groq	A company that hosts open LLMs on fast hardware and provides a free API.
Streamlit	A Python library for building web apps.
LangChain	A Python framework with building blocks for LLM and RAG apps.